

Week 05

Constrained-Random Verification

Jesus Sanchez

Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

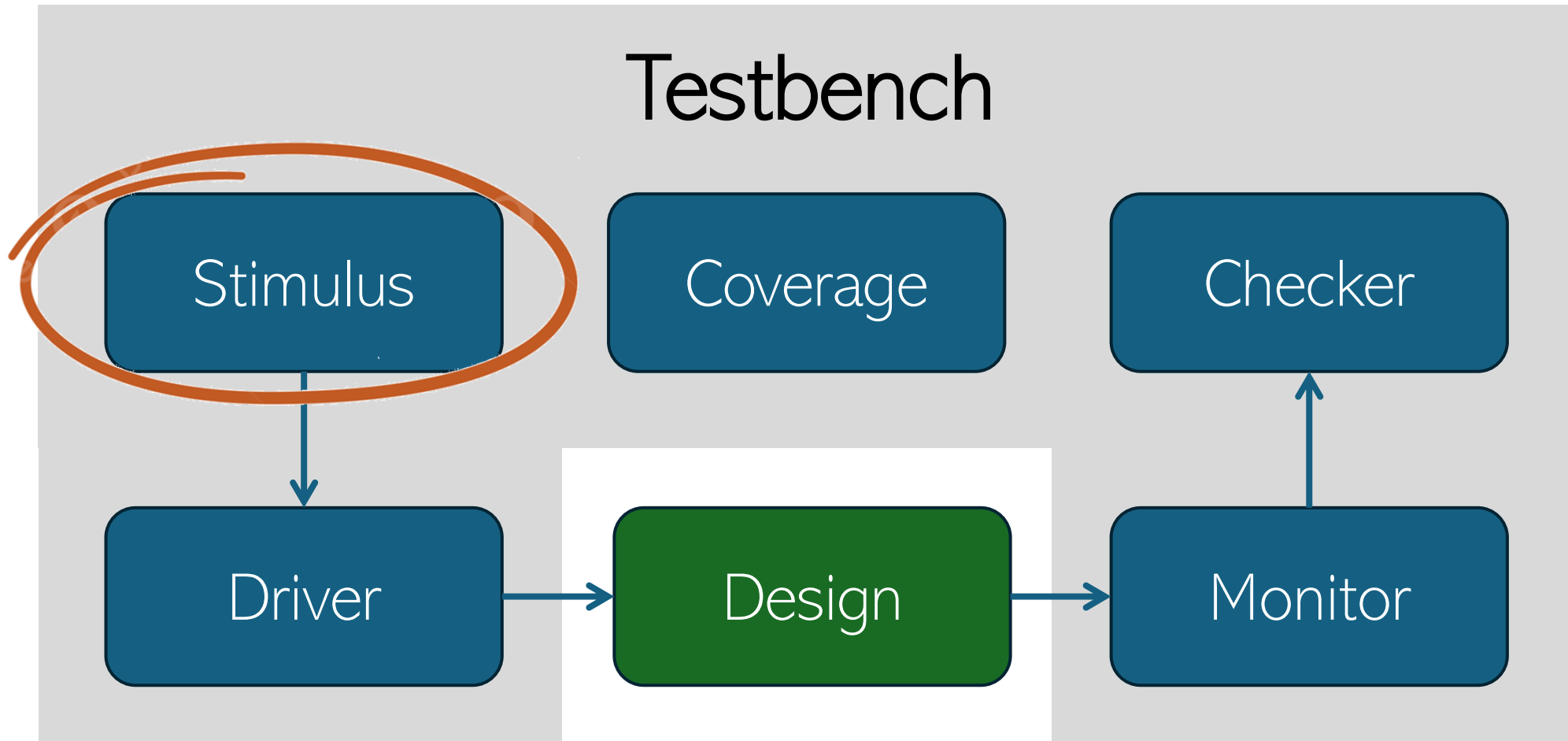
Objectives of the Week

- Stimulus strategies and the role of constrained-random testing
- Practical randomization techniques for generating meaningful variability
- SystemVerilog constraint constructs and how they guide the solver

Agenda

1. Stimulus strategies
2. Randomization techniques
3. SystemVerilog constraints
4. Examples and exercises
5. Wrap-up and self-learning

Basic Testbench Architecture



Stimulus Strategies

Directed
Tests

Random
Tests

Constrained
Random
Tests

Coverage-
Driven
Verification

Table/Graph-
Based
Stimulus

Formal-
Assisted
Stimulus

Mutation
Testing

Directed Tests

■ What?

- Assign specific values to DUT inputs

■ Use Cases:

- Basic functionality
- Small designs, exhaustive (all) combinations
- Known bugs need to be verified as fixed
- Specific corner cases or edge cases
- Verify known critical paths

■ Limitations

- Poor coverage
- Non-scalable
- Tests are biased:
Bugs outside this scope may remain hidden
- **Maintenance burden:** As the design changes, directed tests often need frequent updates.

```
1  ✓ module regfile4(input logic clk, write_enable,
2      |             |             |             input logic [1:0] read_addr, write_addr,
3      |             |             |             input logic [7:0] write_data,
4      |             |             |             output logic [7:0] read_data);
5      |             |             |             logic [7:0] regs [3:0];
6  ✓   always_ff @(posedge clk)
7      |         if(write_enable) regs[write_addr] <= write_data;
8      |         assign read_data = regs[read_addr];
9      endmodule
10
11  ✓ module directed_test;
12      |     logic clk,we;
13      |     logic [1:0] ra,wa;
14      |     logic [7:0] wd,rd;
15
16      |     regfile4 dut(clk,we,ra,wa,wd,rd);
17
18  ✓   initial begin
19      |       clk=0; we=1; wa=0; wd=8'hAA; #1 clk=1;
20      |       #1 $display("%b",rd);
21      |       we=1; wa=1; wd=8'h55; clk=0; #1 clk=1; ra=1;
22      |       #1 $display("%b",rd);
23      |     end
24  endmodule
```

Random Tests

- What?
 - Assign random values to DUT inputs
- Strengths
 - Explores unexpected scenarios
 - High coverage potential
 - Automation-friendly
 - Fewer assumptions: Tests are not biased
- Pitfalls
 - Debugging is harder:
Hard-to-reproduce conditions
 - Coverage gaps:
Critical scenarios might never be tested
 - Over-randomization:
Unlikely or illegal input combinations
 - Unpredictability:
Regression analysis and bug tracing more complex

```
11  ✓ module random_test;
12      logic clk,we;
13      logic [1:0] ra,wa;
14      logic [7:0] wd,rd;
15
16      regfile4 dut(clk,we,ra,wa,wd,rd);
17
18  ✓      initial repeat(5) begin
19          ra=$urandom_range(0,3); wa=$urandom_range(0,3);
20          wd=$urandom_range(0,255); we=$urandom_range(0,1);
21          clk=0; #1 clk=1; #1
22          $display("%b %b -> %b",wa,ra,rd);
23      end
24  endmodule
```

Constrained-Random Tests

- What?
 - Random stimulus guided by rules
- Typical Use Cases
 - Valid instruction sequences in a CPU
 - Memory accesses within legal address space
 - Interrupts under defined timing conditions
 - Cache transactions with proper handshaking
- Benefits
 - Exploration of valid design state space
 - Avoids invalid/illegal inputs
- Challenges
 - Complex or contradictory constraints
 - Coverage-driven tuning is often needed

```
11  module constrained_random_test;
12      logic clk,we;
13      logic [1:0] ra,wa;
14      logic [7:0] wd,rd;
15
16      regfile4 dut(clk,we,ra,wa,wd,rd);
17
18  class rf_input;
19      rand logic [1:0] ra,wa;
20      rand logic [7:0] wd;
21      rand logic we;
22      constraint no_same { ra != wa; }
23  endclass
24
25      rf_input inp=new();
26
27  initial repeat(5) begin
28      if(inp.randomize()) begin
29          ra=inp.ra; wa=inp.wa; wd=inp.wd; we=inp.we;
30          clk=0; #1 clk=1; #1
31          $display("%b %b -> %b",wa,ra,rd);
32      end
33  end
34  endmodule
```

Random vs Constrained-Random

Random Testing

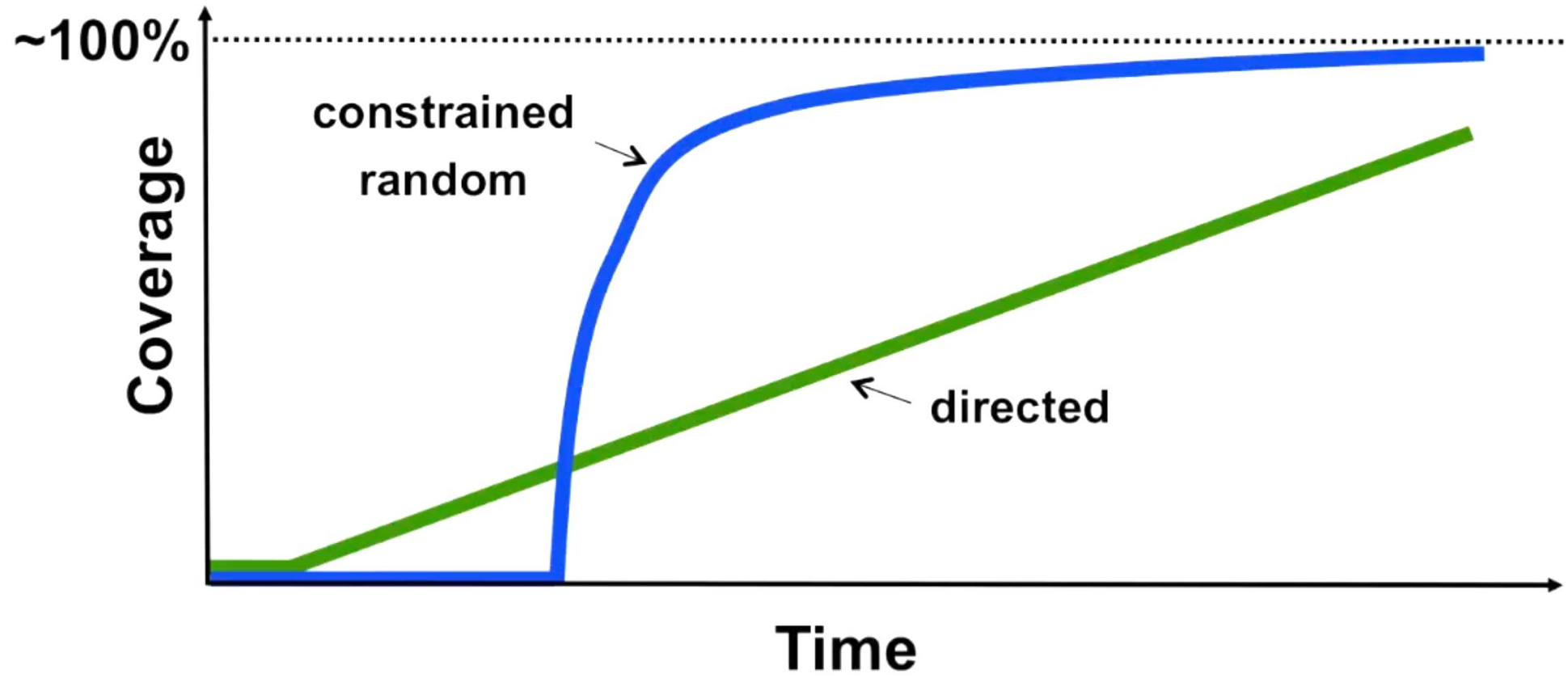
- Fully unrestricted, often results in invalid inputs
- Good for stress test or catching unexpected
e.g., 2-bit opcode 0-7, though only supports 0-3



Constrained Random Testing

- Randomly, but within defined constraints
- Legal, meaningful, or likely scenarios
e.g., pack length, between 64 and 128 bytes

Design Stimulus Techniques vs Coverage



Reproducibility of Random Tests: Seeds

- Why Reproducibility Matters

- Random tests must be repeatable to debug failures
- Without reproducibility, a bug found today may never reappear

- What is a Seed?

- A seed initializes the pseudo-random number generator (PRNG)
- Same seed → same sequence of random values

- Example:

```
initial begin
  counter_transaction trx;
  $urandom(seed);
  trx = new();
  void'(trx.randomize() with { enable == 1; });
end
```

- Or from command line:

```
$ vsim +ntb_random_seed=42
```

Best Practice: Print seed at test start (Seed: %0d, \$urandom)

Agenda

1. Stimulus strategies
2. Randomization techniques
3. SystemVerilog constraints
4. Examples and exercises
5. Wrap-up and self-learning

Randomization Methods

1. Procedural/Function Randomization

- `$random/$urandom/$urandom_range`
- Use in simple assignments (e.g., loop counts, delay values, etc.)

2. Inline (or Scope) Randomization

- `std::randomize`
- Use inside module or simple functions

3. Object/Class Randomization ← This is the most powerful

- `object.randomize()` → `rand/randc`
- Use inside classes using **constraint** statements

Inline and object methods can include constraints

Direct Randomization: Inline or Procedural

PROCEDURAL

```
int signed val;  
val = $random;
```

PROCEDURAL

```
bit [7:0] addr;  
addr = $urandom();           // Uniform random 32-bit value  
addr = $urandom_range(0, 255); // Random value in range
```

INLINE

```
int x;  
std::randomize(x) with { x > 10; x < 100; };
```

INLINE

```
int vals[$];  
for (int i = 0; i < 10; i++) vals.push_back(i);  
vals.shuffle(); // Random order
```

Declaring Random Variables: Object/Class

■ Purpose

- Control randomization of variables for stimulus generation
- Key for random and constrained-random testing

■ Keywords

- **rand** : Variable gets a new random value on each **randomize()** call
- **randc** : Variable takes random values in a cyclic permutation (no repeats until all values are used)

■ Key Takeaways

- Use **rand** when you want complete randomness
- Use **randc** to cycle through all possible values before repeating
- Combine with **constraints** to target valid scenarios.

Randomization with Constraints

■ Purpose

- Define rules or limits for random values
- Generate meaningful, valid transactions rather than purely random data

■ Key Idea

- Use constraint blocks inside classes:

```
class packet;  
  rand bit [7:0] data;  
  rand bit [3:0] addr;  
  
  constraint valid_addr { addr < 10; }  
  constraint data_align { data % 2 == 0; }  
endclass
```



Key Points

- ✓ Constraints guide random values to **valid** and **useful** data
- ✓ Constraints can be **arithmetic** or **logical** expressions
- ✓ If constraints **conflict**, randomization **fails**
- ✓ Crucial for **constrained-random testing**

Clarity and readability

Debugging

Enabling/disabling: `p.constraint_mode(0, "valid_addr");`

Randomization Techniques

10:00

Agenda

1. Stimulus strategies
2. Randomization techniques
3. SystemVerilog constraints
4. Examples and exercises
5. Wrap-up and self-learning

Constraint Syntax

- A constraint statement:

```
constraint constraint_name {  
    constraint;  
    [constraint];  
    [constraint];  
    [...]  
}
```

- A constraint statement can have as many constraints inside as desired
 - Put together by semantics of the name
- We can have as many constraint statements as needed

1. Basic Constraint Type

■ Simple Expression Constraints

- ==, !=, <, >, <= and >=
- E.g., `x > 100 && x <= 150;`

■ Inside Constraints

- Keyword: `inside`
- E.g., `x inside {[0:100], 150};`

■ Inverted Inside

- Keyword: `! inside`
- E.g., `!(x inside {[0:5], [10:20]});`

```
class packet;
  rand bit [7:0] addr;
  rand bit [7:0] length;

  constraint valid_range {
    // Both conditions must be true
    addr >= 10 && addr <= 200;

    // Combined Logic: Length must be even AND greater than 0
    (length % 2 == 0) && (length > 0);
  }
endclass
```

Can use arithmetic operators (+, -, *, /, %, etc.)

2. Probabilistic and Distribution Constraints

- **Distribution Constraints:** Control how often certain values are chosen
 - Keyword: **dist**
 - **:=** Operator
 - Specifies weight to every single value
 - E.g.,
 - **:/** Operator
 - Divides total weight equally among all values
 - E.g.,

```
class packet;
  rand bit [2:0] addr;

  constraint addr_dist {
    // 0 gets weight 20
    // 1, 2, 3 each get weight 50 (Total for range = 150)
    // 4 gets weight 30
    // Total weight = 20 + 150 + 30 = 200
    addr dist { 0 := 20, [1:3] := 50, 4 := 30 };
  }
endclass
```

```
class controller;
  rand bit [2:0] mode;

  constraint mode_dist {
    // 0 gets weight 20
    // 1, 2, 3 share weight 60 (Each gets 60/3 = 20)
    // 4 gets weight 20
    // Total weight = 20 + 60 + 20 = 100
    mode dist { 0 :/ 20, [1:3] :/ 60, 4 :/ 20 };
  }
endclass
```

3. Conditional and Control Constraints

- Implication (->): “one-way” logical rule
 - E.g.; `is_write -> (byte_en != 0);`
- If-Else Constraints: unlike implication it allows an else

• E.g.,

- Solve-Before: order control
 - E.g.,

```
constraint logic_c {  
    (a == 1) -> (b == 0);  
}  
solve a before b;
```

```
constraint length_cfg {  
    if (is_short_pkt) {  
        len inside {[1:64]};  
    } else {  
        len inside {[65:255]};  
        len % 4 == 0; // Large packets must be word-aligned  
    }  
}
```

→ 64 legal values

→ 48 legal values

Probability (The Human's Job): `solve-before` is only used to guide the **likelihood** of results. Without it, the solver treats every valid *combination* of `(len, is_short_pkt)` as equally likely.

The solver never fails because of the order
(only in constraints are contradictory)

4. Advanced and Structural Constraints (1/4)

■ Soft Constraints:

- Keyword: `soft`

```
class packet;
    rand bit [7:0] addr;

    constraint default_addr {
        soft addr == 0; // Default to 0 unless overridden
    }
endclass

// Usage in testbench:
// pkt.randomize() with { addr == 10; }; Inline constraint
// The hard '10' wins; if addr wasn't soft, this would be a failure.
```

4. Advanced and Structural Constraints (2/4)

- Iterative Constraints:
 - Keyword: **foreach**

```
class memory_block;  
    rand bit [31:0] data_array [10];  
  
    constraint valid_data {  
        foreach (data_array[i]) {  
            data_array[i] > 0;           // No element can be 0  
            if (i > 0)  
                data_array[i] > data_array[i-1]; // Each element greater than  
        }  
    }  
endclass
```

4. Advanced and Structural Constraints (3/4)

- Unique Constraints:
 - Keyword: `unique`

```
class scheduler;  
    rand bit [3:0] priorities [4];  
    rand bit [3:0] special_id;  
  
    constraint unique_ids {  
        // All 4 elements in priorities AND special_id must have different  
        unique {priorities, special_id};  
    }  
endclass
```

4. Advanced and Structural Constraints (4/4)

■ Static Constraints:

- Keyword: `static`
- Standard constraints are per-object.
- A static constraint applies to every instance of that class.
- If you change it for one object, it changes for all.

■ Inline Constraints:

```
// Inside the test program:  
initial begin  
    my_pkt.randomize() with { addr inside {[20:30]}};  
end
```

5. Randomization Callbacks

■ What Are They?

- `pre_randomize()` :Called before randomization starts
- `post_randomize()`:Called after randomization finishes

■ Why Use Them?

- Initialize variables before randomization
- Adjust or check values after randomization
- Debug or log randomization process

```
class Packet;
  rand bit [7:0] addr;
  rand bit [7:0] data;

  function void pre_randomize();
    $display("Starting randomization...");
  endfunction

  function void post_randomize();
    $display("addr=%0h data=%0h", addr, data);
  endfunction
endclass
```

Agenda

1. Stimulus strategies
2. Randomization techniques
3. SystemVerilog constraints
4. Examples and exercises
5. Wrap-up and self-learning

Interview Example

- Write a constraint to have a variable divisible by 5.

```
1 | constraint c_div5 { my_variable % 5 == 0; }
```

- Randomize a delay to be between 100 and 500

```
1 | `timescale 1ns/1ps
2 |
3 | module tb;
4 |     initial begin
5 |         int delay;
6 |
7 |         std::randomize(delay) with { delay inside {[100:500]}; };
8 |         #(delay) $display("Statement printed after %0d delay", delay);
9 |
10 |        // This is also good enough, although there's no variable to print actual randomized
11 |        #($urandom_range(100, 500)) $display("Some delay between 100 to 500");
12 |    end
13 | endmodule
```

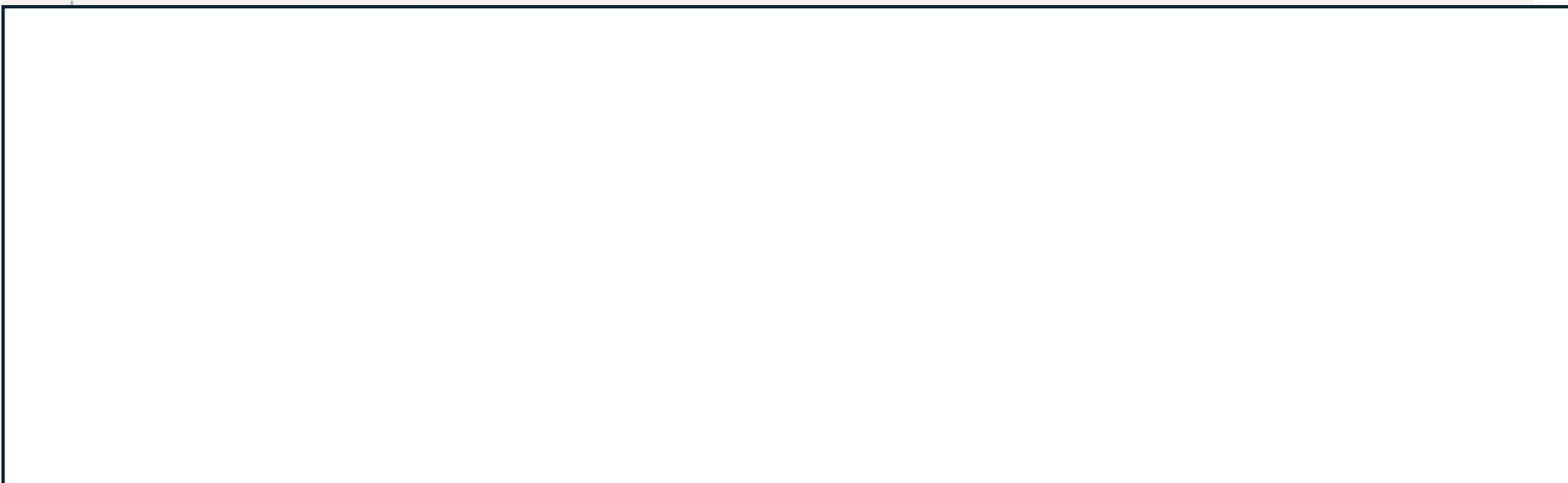
Interview Example

- Assume a memory region exists from 0x2000 to 0x4000 that is byte addressable. Write SV constraints to randomly pick an address within this memory region that is aligned to 4-byte boundary.

```
1 | bit [31:0]      addr;  
2 | constraint c_addr {   addr inside {[32'h2000:32'h4000]};  
3 |                     addr % 4 == 0;  
4 |                     // addr[1:0] == 0;      Also okay  
5 |                     }
```

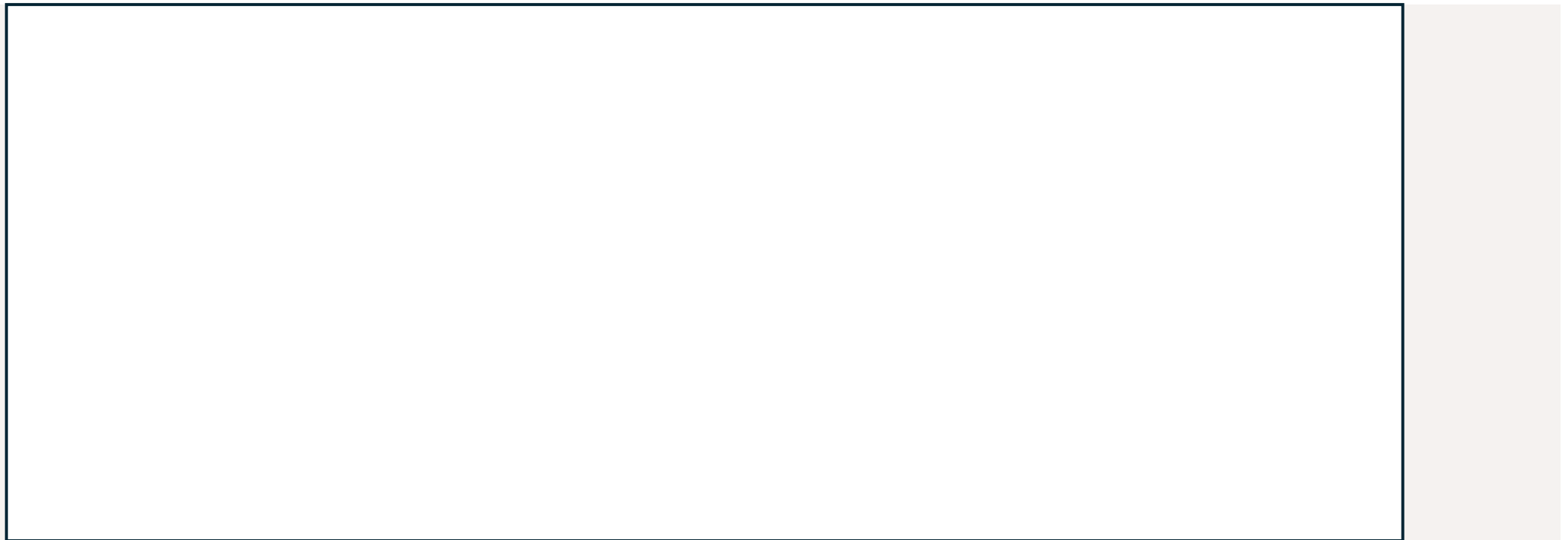
Interview Question

- Assume memory region from 0x0 to 0x100. There is a small region in between from 0x20 to 0xE0 that is reserved. Write SystemVerilog constraints to choose a block of memory of size 16 bytes that is outside the reserved region and inside the entire memory range. The starting address of the block should be 4-byte aligned.



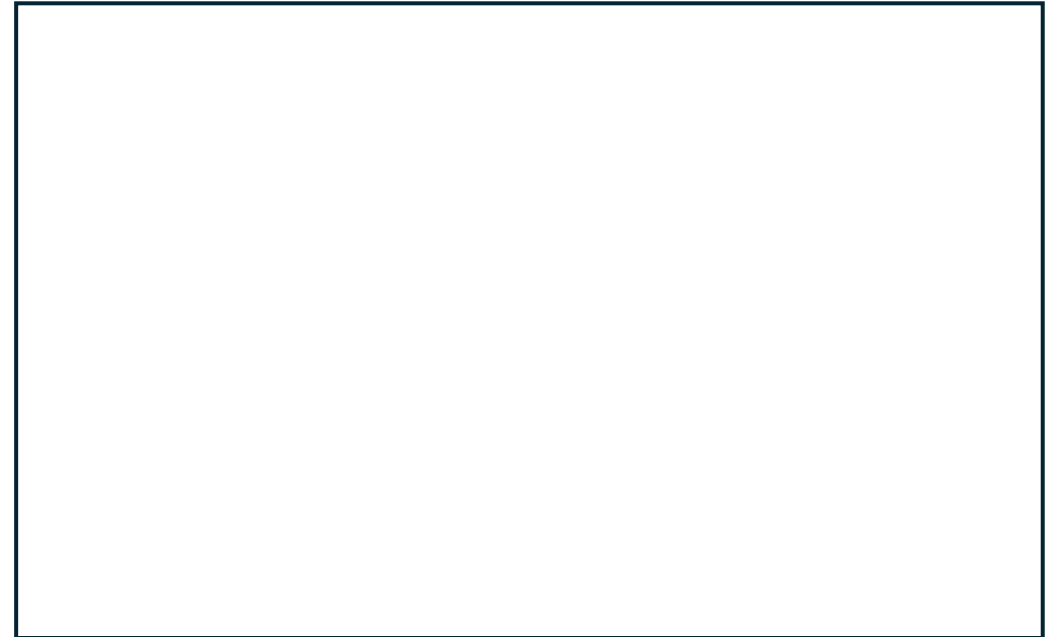
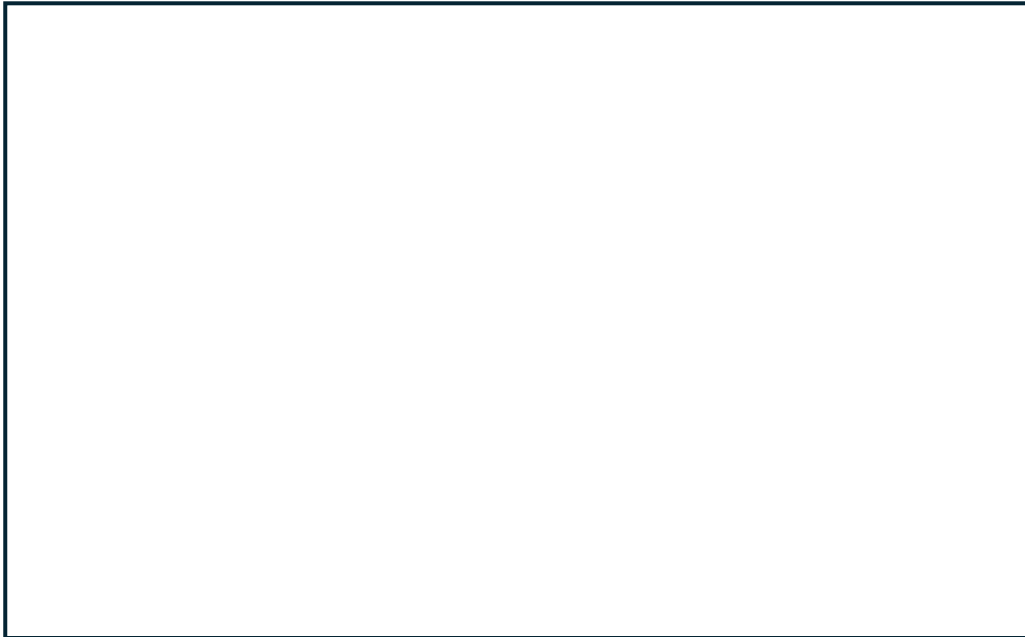
Interview Question

- Constrain a dynamic array such that it does not pick values from another array.



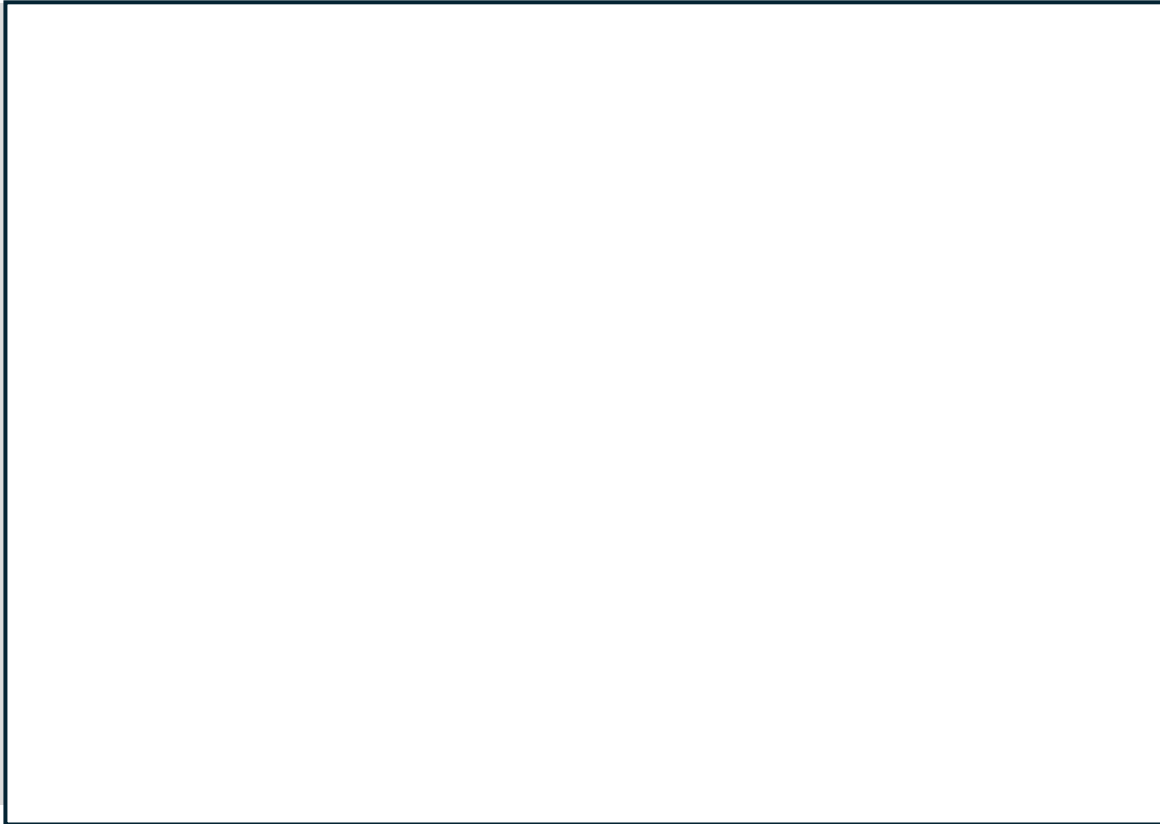
Interview Question

- How do you constrain an array to have all unique elements?
- How do you constrain an array elements to be in ascending order?



Interview Question

- How do you create weighted random array with specific element probabilities?



Agenda

1. Stimulus strategies
2. Randomization techniques
3. SystemVerilog constraints
4. Examples and exercises
5. Wrap-up and self-learning

Summary of Key Concepts from this Session

- Reviewed key stimulus strategies and how constrained-random verification improves coverage
- Applied SystemVerilog constraint constructs to shape and control stimulus
- Practiced multiple randomization techniques through guided examples

Link to Next Week

■ Week 06: Assertion-Based Verification

- Introduction to assertions and their role in modern verification
- SystemVerilog Assertions (SVA) fundamentals and syntax
- Temporal properties and checking protocol behavior
- Integrating assertions into simulation and formal workflow

Self-Learning Corner

■ Books

- SystemVerilog for Verification (3rd Ed.) by Chris Spear
 - Chapter 6: Randomization

■ Videos

- Directed vs Random Testing:
<https://www.youtube.com/watch?v=fkyu88hMwwwo&t=22s>

■ Articles

- <https://theartofverification.com/directed-testing-vs-constraint-random-verification>

“Randomness is not a substitute for thinking — it’s a tool for discovering what thinking alone would miss.”

Common saying in functional verification